Manual operativo de datos — Ecosistema de consultas comerciales de Okinet

1. Objetivo y funcionamiento

El sistema utiliza Gmail para recibir consultas y enviar comunicaciones, Airtable para almacenar información y Groq para validar consultas y generar borradores. Make coordina las integraciones mediante dos escenarios.

Escenario 1 — Recepción y generación

Gmail recibe consultas → Airtable busca registros previos → registra la consulta → recupera información de Base RAG Okinet → Groq procesa el correo → un Router separa consultas completas e incompletas.

- Cuando la consulta contiene el volumen mensual de impresiones, guarda el borrador con Estado **En revisión** y envía un aviso interno.
- Cuando falta ese dato, cambia el Estado a **Rechazado** y envía una notificación interna.

Escenario 2 — Envío aprobado

Airtable busca consultas aprobadas y pendientes de envío → Gmail envía el borrador aprobado al cliente → Airtable cambia el Estado a **Publicado** y registra la fecha de envío.

La conexión entre ambos escenarios es el registro compartido en **Centro de Comando**.

2. Esquema de tablas vinculadas

La base **entrega final** contiene dos tablas.

Centro de Comando

Campo	Estructura	Función
Asunto	Texto	Asunto original del correo.
Nombre	Texto	Nombre del remitente.
email	Dirección de correo	Destinatario de la respuesta.
consulta	Texto	Contenido completo de la consulta.
ID del mensaje	Texto	Identificador del mensaje de Gmail.
thread id	Texto	Identificador del hilo original.
Estado	Selección única	Seguimiento del procesamiento.

Aprobado	Booleano	Autorización humana para enviar.
Fecha de ejecución	Fecha y hora	Momento de procesamiento.
Fecha de envío	Fecha y hora	Momento del envío al cliente.
borrador generado	Texto	Respuesta de IA, editable antes de aprobar.
Responsable de revisión	Objeto de colaborador	Identificación de la persona revisora.
detalle del error	Texto	Registro de incidencias.
Base relacionada	Lista de IDs de registros	Vínculo con la información RAG.
Fecha de creación	Fecha	Metadato del registro.
Última modificación	Fecha	Metadato del registro.

Base RAG Okinet

Campo	Estructura	Función
Tema	Texto	Servicio o tema documentado.
Contenido	Texto	Información de referencia para la IA.
Categoría	Clasificación	Organización del conocimiento.
Palabras clave	Texto	Términos asociados al tema.
Fecha de actualización	Fecha	Control de vigencia.
Nivel de confidencialidad	Clasificación	Identificación del acceso a la información.
Centro de Comando	Lista de IDs de registros	Relación inversa con las consultas.

Relación: los campos **Base relacionada** y **Centro de Comando** permiten vincular las consultas con el conocimiento utilizado. Sin embargo, los Blueprints actuales no completan esa relación automáticamente. Debe mapearse el ID del registro recuperado por Airtable 4 en **Base relacionada**.

3. Estados y aprobación humana

Estado	Cómo se establece
Generando	Escenario 1, al crear la consulta.

En revisión	Escenario 1, al guardar el borrador.
Rechazado	Escenario 1, cuando falta el volumen mensual.
Aprobado	La persona revisora lo selecciona después de revisar el texto.
Publicado	Escenario 2, después del envío correcto.

Para habilitar el envío, la persona revisora debe **marcar Aprobado y seleccionar Estado = Aprobado**. La existencia de un borrador por sí sola no autoriza el envío.

4. Esquemas JSON de transferencia

Los ejemplos siguientes muestran las estructuras y los campos utilizados. Los valores son ficticios y se omiten metadatos. Para facilitar la lectura, los campos de Airtable aparecen por nombre; los Blueprints utilizan identificadores internos `fld…` al escribirlos.

4.1. Gmail → Make → Airtable

```
{
  "fields": {
    "Asunto": "Consulta por alquiler de fotocopiadora",
    "Nombre": "María Pérez",
    "email": "cliente@example.com",
    "consulta": "Necesitamos una fotocopiadora para 2000 impresiones mensuales.",
    "ID del mensaje": "mensaje_ejemplo_001",
    "thread id": "hilo_ejemplo_001",
    "Estado": "Generando"
  }
}
```

El escenario 1 mapea `subject`, `fromName`, `fromEmail`, `fullTextBody`, `id` y `threadId` del módulo Gmail a esos campos.

Gmail consulta **Inbox**, recupera el contenido completo, tiene un límite de **10 mensajes** por ejecución y no los marca como leídos.

4.2. Airtable → Groq

```
{
  "model": "openai/gpt-oss-20b",
  "messages": [
    {
      "role": "system",
      "content": "Instrucciones comerciales y validación del volumen mensual de impresiones."
    },
    {
      "role": "user",
```

```
      "content": "INFORMACIÓN DE REFERENCIA: contenido de Airtable. ASUNTO DE LA CONSULTA: asunto del correo. CONSULTA DEL CLIENTE: cuerpo completo del correo."
    }
  ],
  "max_tokens": 500
}
```

El mensaje `system` contiene las reglas. El mensaje `user` incorpora dinámicamente:

- `{{4.Contenido}}`: conocimiento recuperado.
- `{{1.subject}}`: asunto del correo.
- `{{1.fullTextBody}}`: consulta del cliente.

La búsqueda RAG está fijada al tema **Alquiler de fotocopiadoras**, con un máximo de **10 registros**.

La salida de Groq está configurada como texto. Si falta el volumen mensual aproximado de impresiones, el prompt solicita devolver únicamente `ERROR_DATOS_INCOMPLETOS`.

4.3. Groq → Router → Airtable

Representación simplificada de la salida utilizada por Make:

```
{
  "result": "ERROR_DATOS_INCOMPLETOS"
}
```

El Router evalúa `{{5.result}}`:

Resultado	Ruta
Diferente de `ERROR_DATOS_INCOMPLETOS`	Guarda el resultado como borrador y cambia el Estado a En revisión.
Igual a `ERROR_DATOS_INCOMPLETOS`	Cambia el Estado a Rechazado.

Para el camino de revisión, se actualiza el registro creado en el módulo 3:

```
{
  "id": "rec_consulta_ejemplo",
  "fields": {
    "Estado": "En revisión",
    "borrador generado": "Texto de respuesta generado por Groq.",
    "Fecha de ejecución": "2026-10-11T14:00:00Z"
  }
}
```

La fecha se obtiene mediante `now`. Después, Gmail envía una notificación interna con el borrador y las instrucciones de aprobación.

4.4. Airtable → escenario 2

El módulo inicial utiliza esta fórmula:

```
AND(
  {Aprobado}=1,
  {Estado}="Aprobado",
  {Fecha de envío}=BLANK(),
  {borrador generado}!=BLANK()
)
```

Las cuatro condiciones deben cumplirse simultáneamente. La búsqueda devuelve hasta **10 registros** por ejecución.

Ejemplo del registro seleccionado:


```
{
  "id": "rec_consulta_ejemplo",
  "email": "cliente@example.com",
  "Asunto": "Consulta por alquiler de fotocopiadora",
  "Aprobado": true,
  "Estado": "Aprobado",
  "borrador generado": "Respuesta revisada y autorizada.",
  "Fecha de envío": null
}
```


`Aprobado` es booleano. La comparación con `1` corresponde a la casilla marcada en la fórmula de Airtable. Los demás filtros verifican el estado y la presencia o ausencia de datos.

4.5. Escenario 2 → Gmail


```
{
  "to": ["cliente@example.com"],
  "subject": "Re:Consulta por alquiler de fotocopiadora",
  "bodyType": "rawHtml",
  "content": "Respuesta revisada y autorizada."
}
```


El módulo Gmail 4 obtiene:

- Destinatario: `{{1.email}}`.
- Asunto: `Re:{{1.Asunto}}`.
- Contenido: el campo **borrador generado** del registro aprobado.

El Blueprint utiliza **Send an Email** y no mapea `thread id`. Por tanto, el envío no tiene configurada explícitamente la continuidad del hilo original.

4.6. Gmail → actualización final en Airtable

Después del envío correcto, Airtable 5 actualiza el mismo registro mediante `{{1.id}}`:

```
{
  "id": "rec_consulta_ejemplo",
  "fields": {
    "Estado": "Publicado",
    "Fecha de envío": "2026-10-11T14:15:00Z"
  }
}
```

El cambio de estado y la fecha hacen que el registro deje de cumplir la fórmula de selección del escenario 2.

5. Operación diaria

1. Revisar el aviso interno del escenario 1.
2. Abrir la consulta en Centro de Comando.
3. Corregir el borrador cuando sea necesario.
4. Marcar **Aprobado** y seleccionar Estado **Aprobado**.
5. Verificar posteriormente que el registro quede **Publicado**, con fecha de envío.
6. Revisar las consultas rechazadas y completar los datos faltantes antes de reprocesarlas.

6. Ajustes pendientes detectados

Los Blueprints muestran estas diferencias que deben resolverse:

- **Duplicados:** la fórmula de Airtable 2 del escenario 1 conserva `"VARIABLE"` como texto literal. Debe mapearse el ID real del mensaje y verificarse la continuación cuando no hay coincidencias.
- **Tablas vinculadas:** falta completar automáticamente **Base relacionada**.
- **Consultas incompletas:** Airtable 9 y Gmail 12 utilizan `{{5.error.message}}`, aunque el rechazo proviene de una respuesta normal de Groq. Debe reemplazarse por el motivo de información incompleta.
- **Aviso de rechazo:** Gmail 12 afirma que el Estado es Error, mientras Airtable registra Rechazado. Deben coincidir.
- **Error Handling:** el Retry del escenario 1 está asociado a Gmail 12, con **3 intentos cada 15 minutos**. Falta un manejador directamente en Groq para registrar fallas de la IA.
- **Hilos:** el escenario 2 no mapea Thread ID. El prefijo “Re:” no sustituye ese mapeo.
- **Formato:** los correos utilizan Raw HTML con texto sin formato HTML. Conviene usar contenido de texto o preparar HTML válido.
- **Envío y registro:** si Gmail envía correctamente pero falla la actualización final de Airtable, la consulta puede seguir apareciendo como pendiente. Debe recuperarse la actualización fallida sin repetir el correo.

- **RAG:** si existen varias coincidencias, debe limitarse la búsqueda o reunirse la información antes de generar un único borrador.

Estos puntos describen la configuración exportada; los Blueprints por sí solos no acreditan ejecuciones exitosas.